Sistemas Distribuidos I
(75.74)

Multicomputing, Comunicaciones y Virtualización

Multicomputing, TCP/UDP, Docker

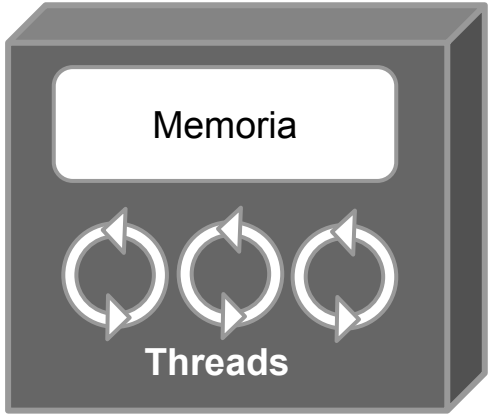
Docentes

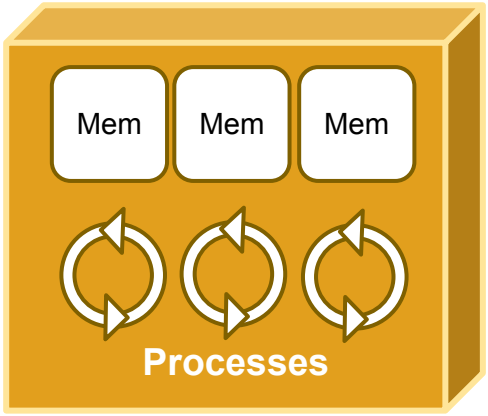
- Pablo D. Roca
- Ezequiel Torres Feyuk

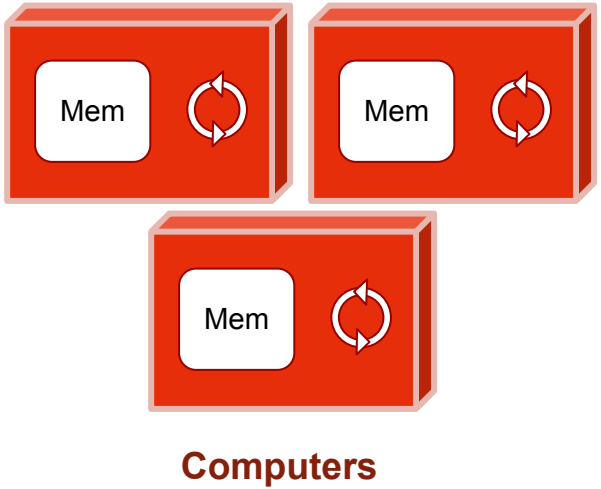
Agenda

- **Multi-Processors & Multi-Computing**
- Comunicaciones
- Virtualización
- Docker & Docker-Compose

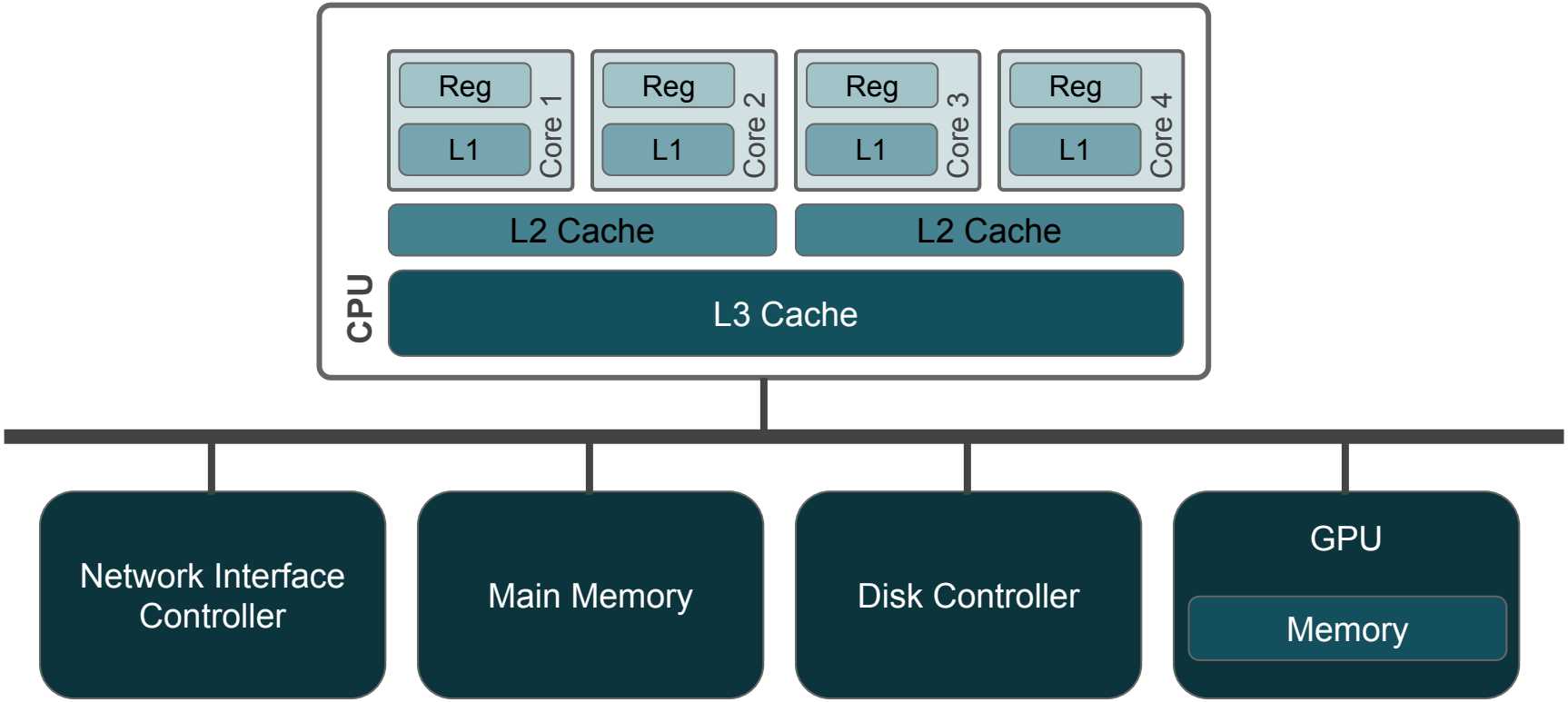
Modelos de Multiprogramación



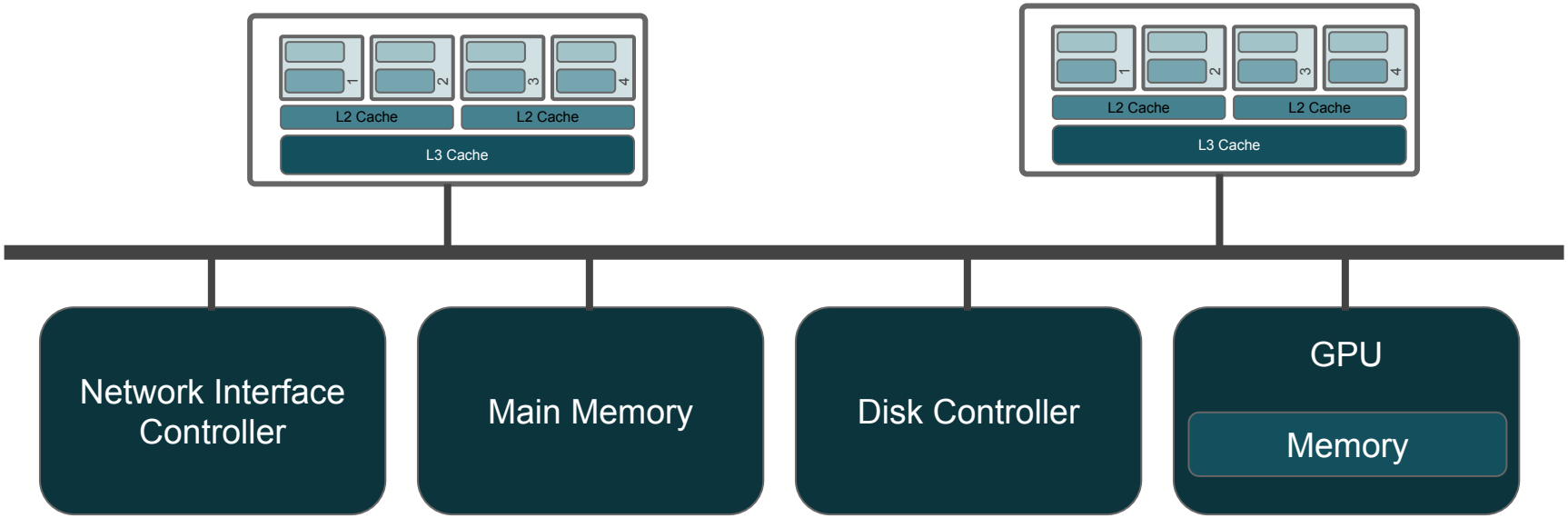




Modelo Físico | Una CPU



Modelo Físico | Varias CPUs



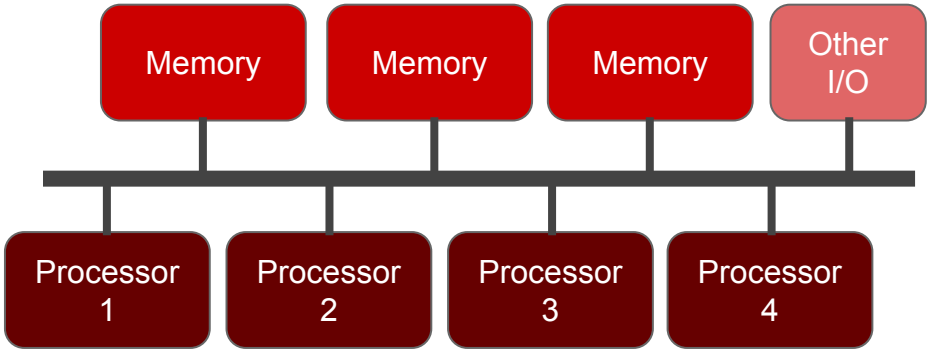
Muchos Procesadores | Taxonomía de Flynn

Clasificación de sistemas de acuerdo a cardinalidad de flujos de instrucciones (procesadores) y flujos de datos (memoria).

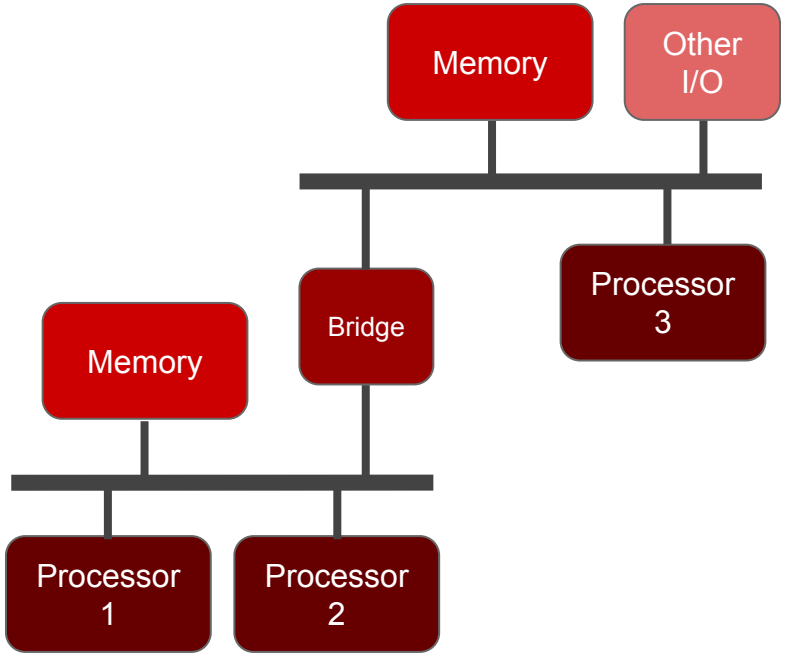
- **SISD (*Single Instruction Single Data*):** modelo estándar de un procesador sin paralelismo
- **SIMD:** *array processors*. Ej: GPU
- **MISD:** no son usuales (procesar múltiples veces los mismos datos de forma determinística retorna el mismo resultado)
- **MIMD:** se divide en dos modelos:
  - *Multiprocessors* (?): con memoria y/o *clock* compartidos
  - *Multicomputers*: sin memoria ni *clock* compartidos

MIMD | *Multiprocessors* (Memoria Compartida)

Symmetric Multiprocessing

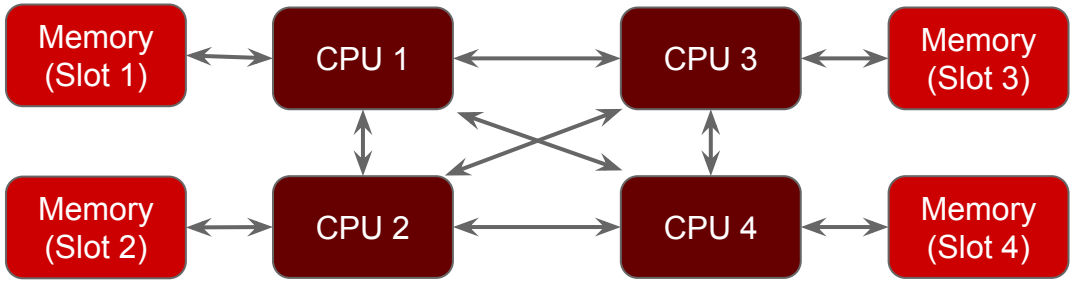


Asymmetric Multiprocessing



MIMD | *Multiprocessors* | UMA vs NUMA

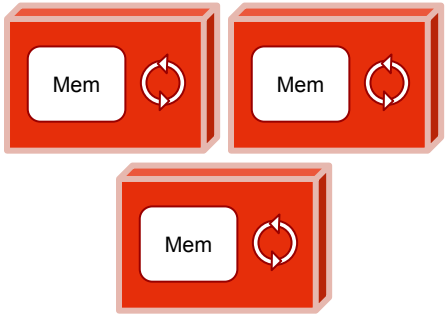
- Uniform Memory Access (**UMA / non-NUMA**)
  - Tiempo de acceso a la memoria es idéntico para todos los procesadores.
- Non Uniform Memory Access (**NUMA**):
  - Ideado en SGI, ahora presente en Linux kernel y MS Servers
  - Cada CPU controla un bloque de memoria y se transforma en su 'home agent'



MIMD | *Multicomputers* (sin memoria compartida)

- Cada computadora tiene su propia memoria local
- Cada computadora puede fallar de forma independiente
- No poseen un reloj central de ejecución de instrucciones
- Requieren comunicación entre computadoras:
  - Networking: LAN, MAN, WAN

Multicomputing



Recursos Compartidos

- Ninguno

Sincronización

- Mensajes Ad-Hoc entre Computadoras
  => necesidad de implementar mecanismos de sincronización.

Características clave

- Comunicación de red => problemas por limitaciones de ancho de banda, latencia y pérdida de mensajes.
- Comunicación entre procesos: compleja y central al diseño del sistema.
- Alta escalabilidad y tolerantes a fallos.

Agenda

- Multi-Processors & Multi-Computing
- **Comunicaciones**
- Virtualización
- Docker & Docker-Compose

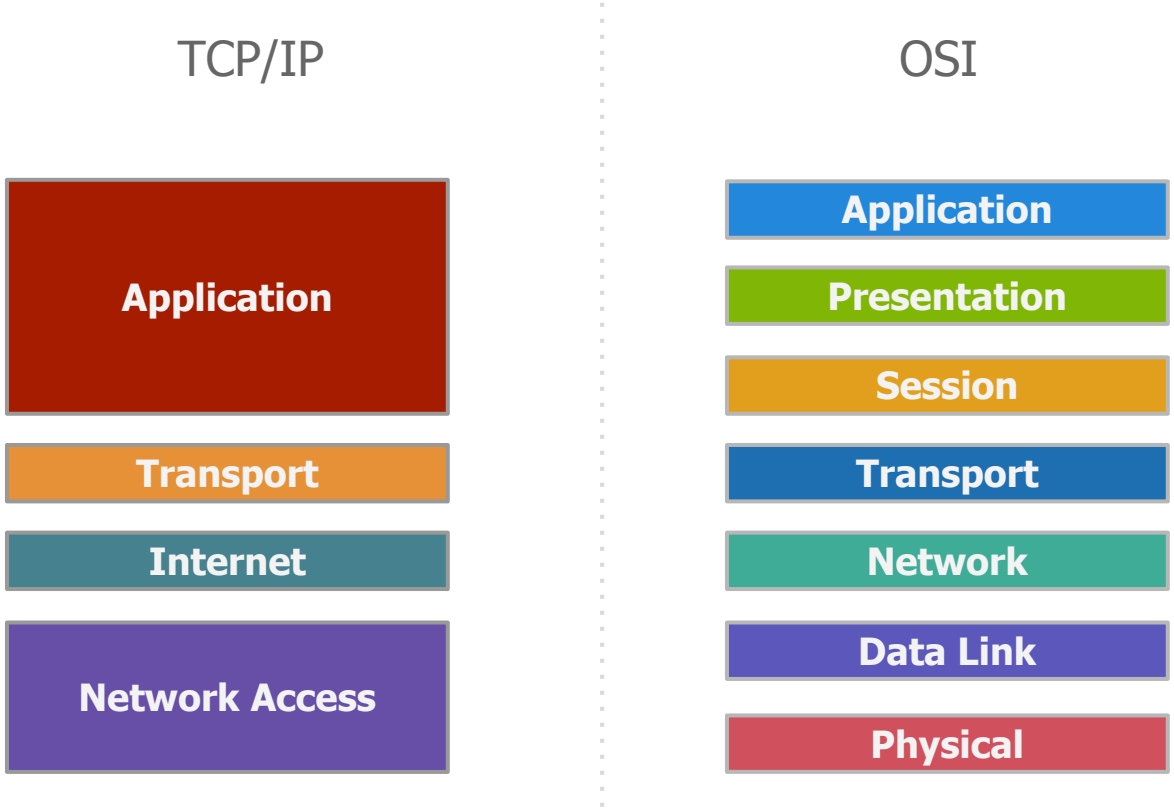
Comunicaciones | Modelo TCP/IP

Application	Aplicaciones de usuario, representación de datos y
Transport	Comunicación punto a punto
Internet	Lógica de transmisión de datos sobre la red
Network Access	Transferencia física confiable, libre de errores.

Comunicaciones | Modelo OSI

Application	Aplicaciones de usuario
Presentation	Representación de datos
Session	Manejo de conexiones y sesión
Transport	Transferencia confiable, libre de errores
Network	Establecer, mantener y terminar conexiones. Transmisión
Data Link	Sincronización, control de errores y envío de frames
Physical	Manejo del medio físico para transmitir bits

Comunicaciones | Relaciones entre OSI y TCP/IP



Estructura de Paquetes | IP

<table>
<tr><td>Ver.</td><td>IHL</td><td>Type of Service</td><td colspan="2">Total Length</td></tr>
<tr><td colspan="3">Identification</td><td>Flags</td><td>Frag. Offset</td></tr>
<tr><td colspan="2">TTL</td><td>Protocol</td><td colspan="2">Header Checksum</td></tr>
<tr><td colspan="5">Source Address</td></tr>
<tr><td colspan="5">Dest. Address</td></tr>
<tr><td colspan="5">Options</td></tr>
<tr><td colspan="5">Data (variable)</td></tr>
</table>

32 bits

Estructura de Paquetes | TCP y UDP

TCP

- Orientado a conexión
- Asegura entrega y orden

<table>
<tr><td colspan="2">Source Port</td><td colspan="2">Dest. Port</td></tr>
<tr><td colspan="4">Sequence Number</td></tr>
<tr><td colspan="4">Ack. Number</td></tr>
<tr><td>Data Offset</td><td>Reserv ed</td><td>Flags</td><td>Window (sliding win.)</td></tr>
<tr><td colspan="2">Checksum</td><td colspan="2">Urgent Pointer</td></tr>
<tr><td colspan="3">Options</td><td>Padding</td></tr>
<tr><td colspan="4">Data (variable)</td></tr>
</table>

32 bits

UDP

- Orientado a datos
- Sin garantías: *best effort*

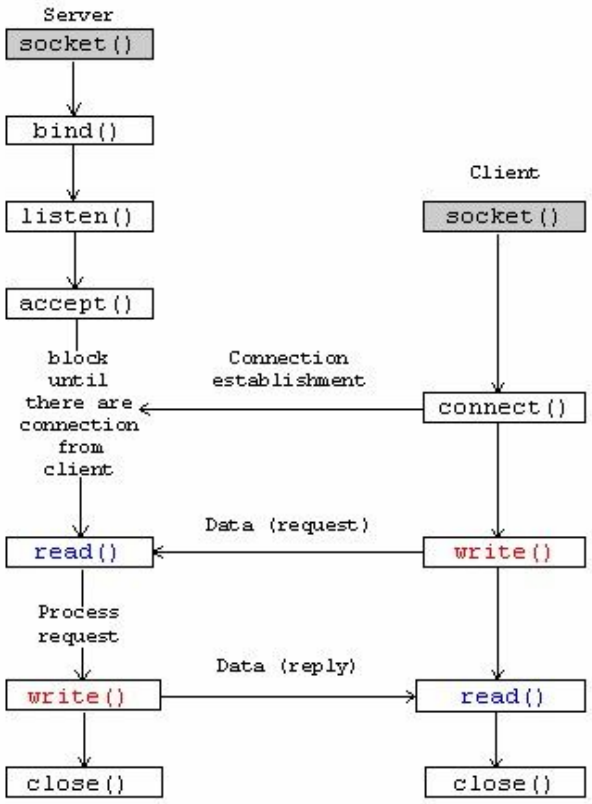
<table>
<tr><td>Source Port</td><td>Dest. Port</td></tr>
<tr><td>Length</td><td>Checksum</td></tr>
<tr><td colspan="2">Data (variable)</td></tr>
</table>

32 bits

Sockets | Primitivas

- `socket(domain: int, type: int, protocol: int): int`
- `bind(fd: int, addr: struct sockaddr*, addrlen: int): int`
- `listen(fd: int, backlog: int): int`
- `accept(fd: int, addr: struct sockaddr*, addrlen: int): int`
- `connect(fd: int, addr: struct sockaddr*, addrlen: int): int`
- `send(fd: int, buf: void*, buflen: int, flags: int): int`
- `receive(fd: int, buf: void*, buflen: int, flags: int): int`
- `close(fd: int): int`

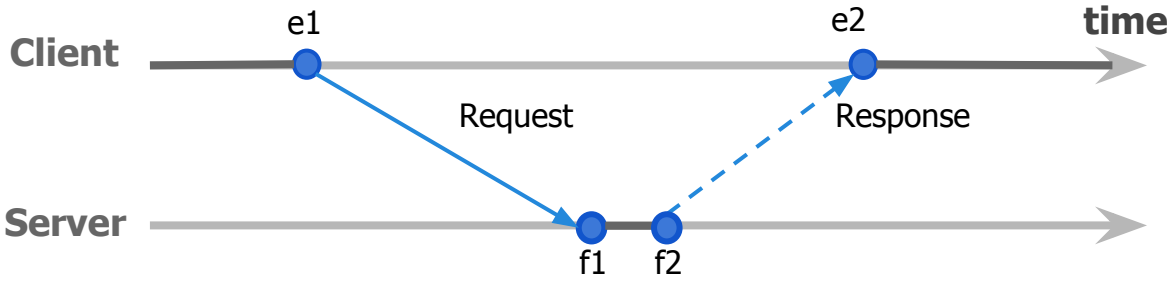
Sockets | Flujo de uso



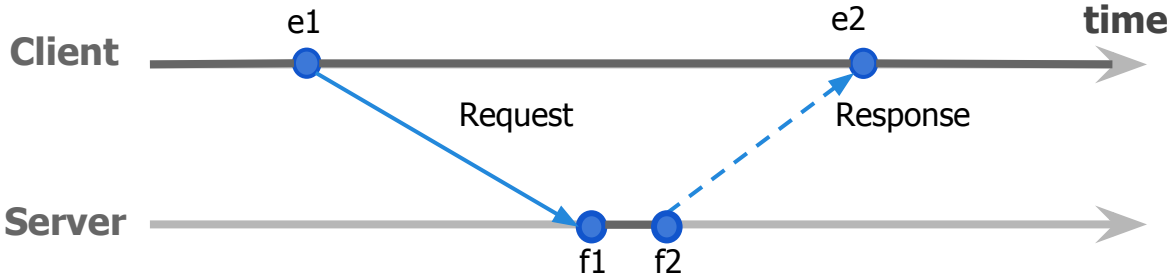
Source: https://stackoverflow.com/questions/27014955/socket-connect-vs-bind

Mensajes Sincrónicos vs Asincrónicos

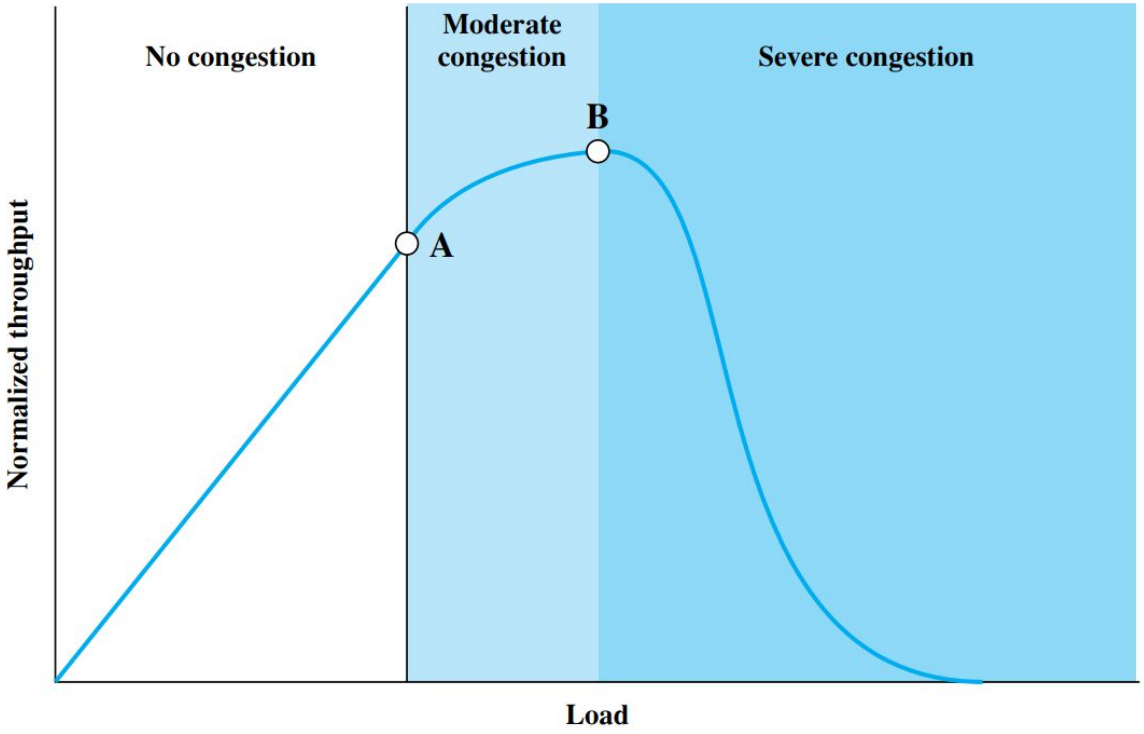
Sincrónico



Asincrónico

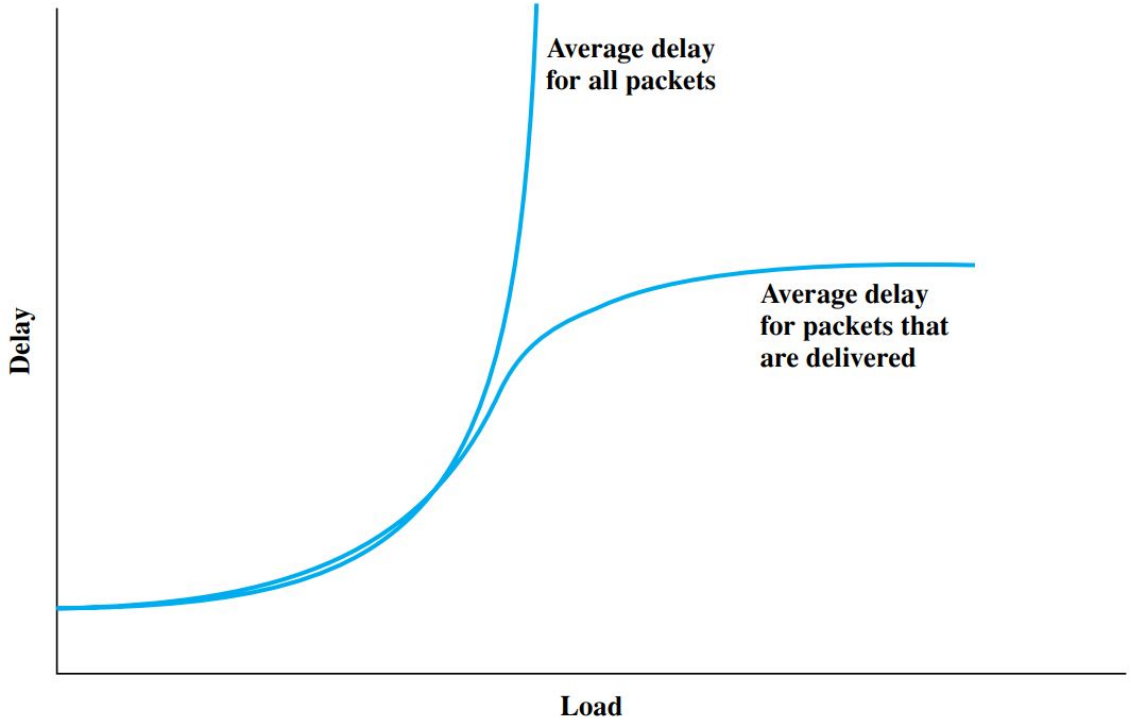


Congestión de Red | *Throughput*



Source: W. Stallings. Data and Computer Communications. 8th edition.

Congestión de Red | *Delay*



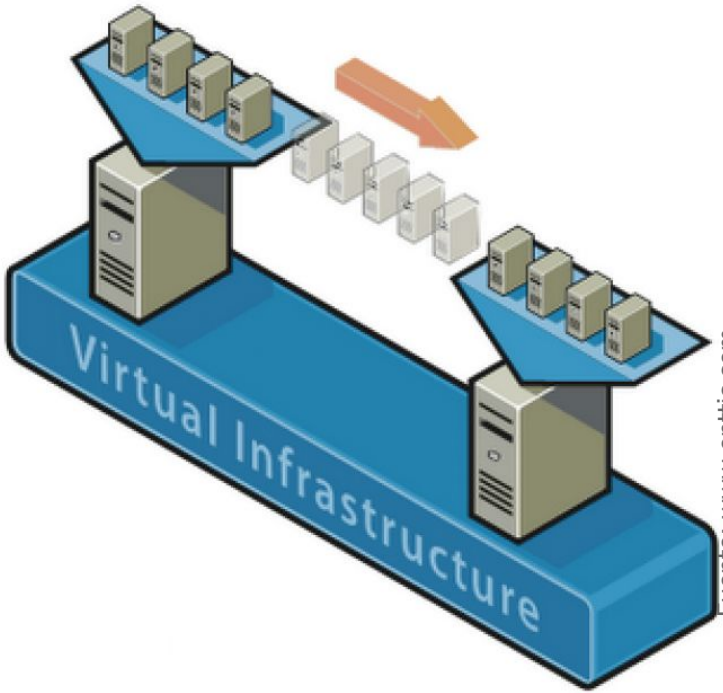
Source: W. Stallings. Data and Computer Communications. 8th edition.

Agenda

- Multi-Processors & Multi-Computing
- Comunicaciones
- **Virtualización**
- Docker & Docker-Compose

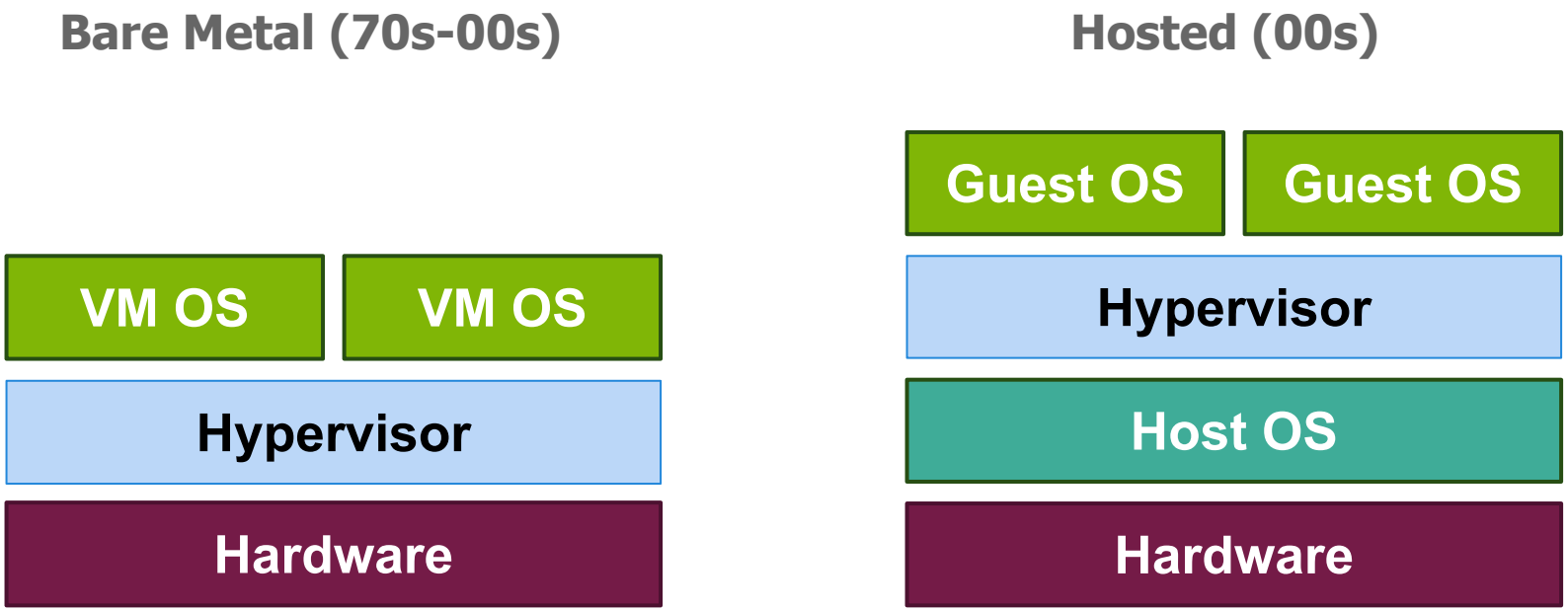
Virtualización

- Necesidad de independencia real de los recursos
  - Seguridad en el acceso
  - Diferenciación entre instrucciones con privilegios vs sin privilegios.
- Hypervisor (Virtual Machine Monitor)
  - Manager de VMs
  - Emulación de Hardware Capabilities
  - Administración de recursos del Host OS hacia los Guest OS
  - Implementación de mecanismos de seguridad



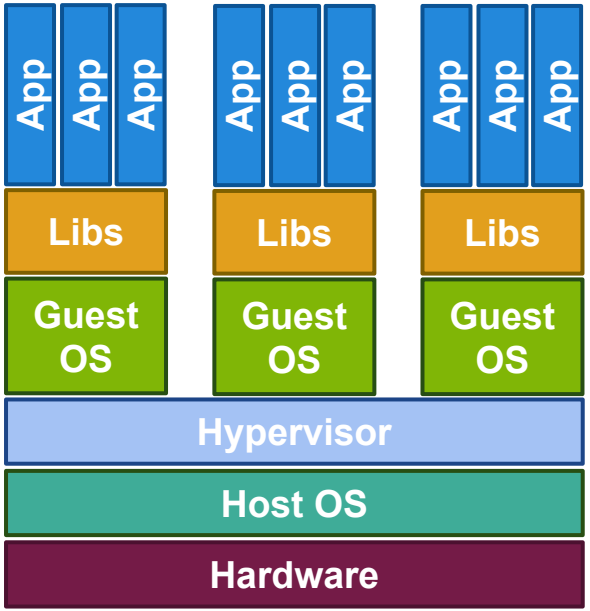
Fuente: www.opttic.com

Virtualización | *Hypervisor*

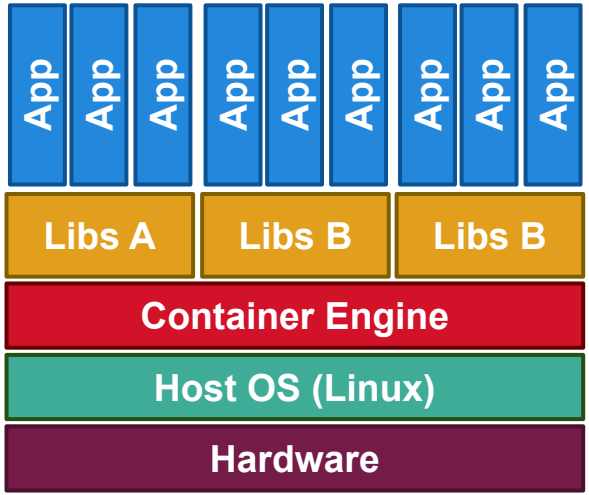


Virtualización | *VMs vs Containers*

Hosted (00s)



Containers (10s)

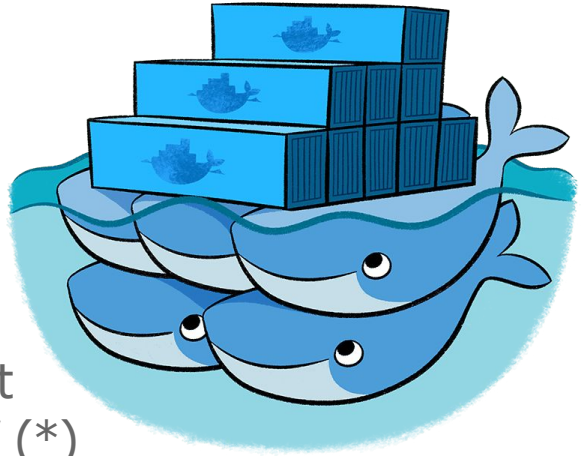


Agenda

- Multi-Processors & Multi-Computing
- Comunicaciones
- Virtualización
- **Docker & Docker-Compose**

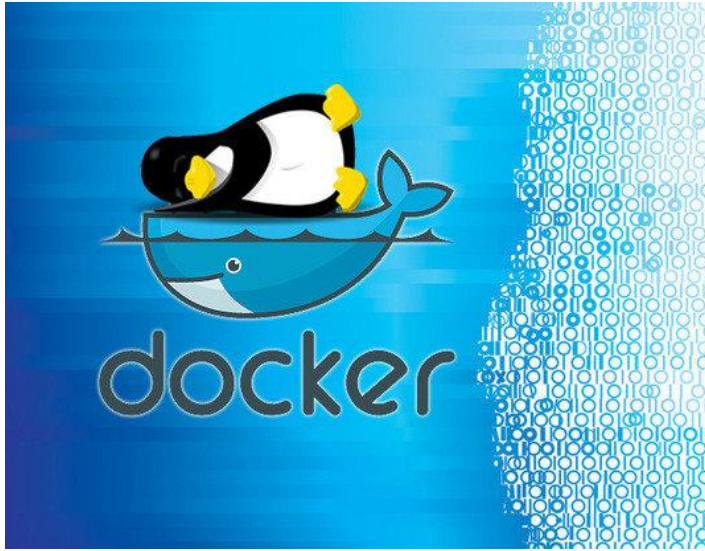
Docker

- Desarrollado en Golang
- Primera versión estable en 2013
- Basado fuertemente en features provistos por el kernel del SO y el Hypervisor:
  - Linux: cgroups, namespaces, union mount
  - Mac: recientemente soportado con HyperKit
  - Win: recientemente soportado con Hyper-V (*)
- Rápida adopción por parte de la industria
  - Kubernetes, Mesos, AWS, GCP, Azure, etc. soportan Docker



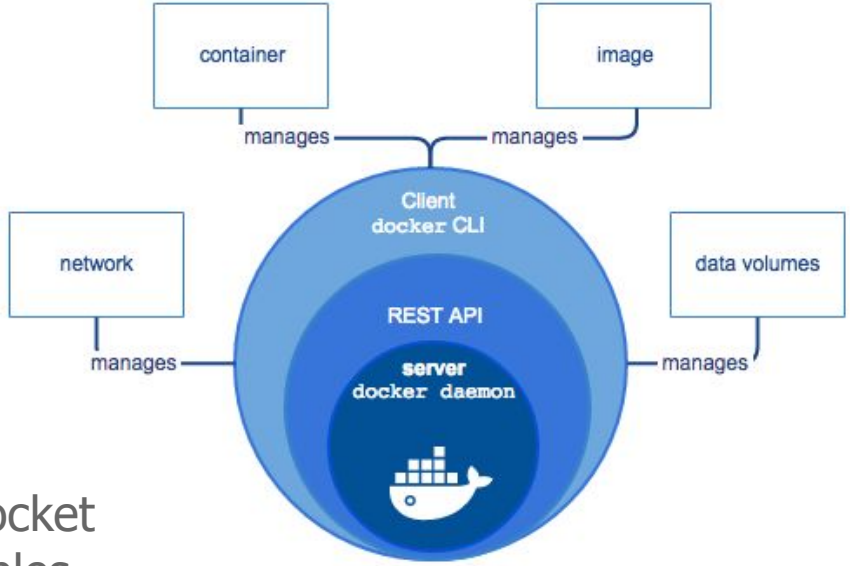
Docker | Soporte de OS

- **Namespaces:** Aislamiento de Recursos
  - NET (Network Stack)
  - PID (Procesos)
  - MNT (Mount)
  - IPC (Semáforos, Shared Memory, etc.)
  - USER (user and group ids)
- **Cgroups**: Seguridad y administración de recursos
- **Union Mount:** Múltiples directorios vistos como uno



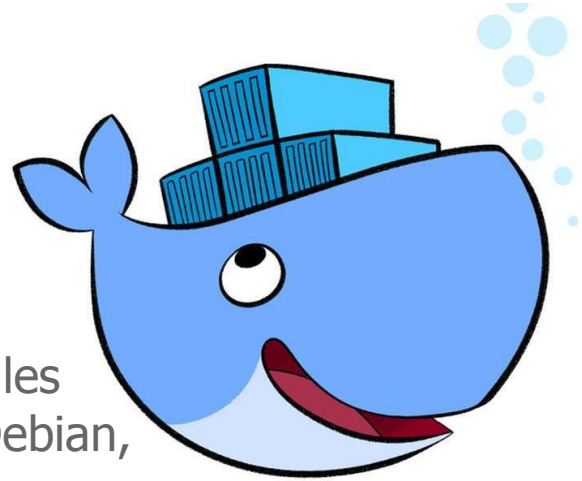
Docker | Engine

- **Docker Daemon**
  - Manejo de todos los recursos (images, containers, volumes, network, etc.)
  - Nueva funcionalidad a través de Plugins (Ej. convoy)
- **Docker REST API**
  - Sumamente flexible
  - Fácil crear nuevos clientes con funcionalidades específicas
- **Docker CLI**
  - Acceso REST API a través de unix socket
  - Gran cantidad de comandos disponibles



Docker | Conceptos Básicos (I)

- **Dockerfile**
  - Secuencia de comandos que generan Images
  - Instalación de dependencias, librerías
  - Cada comando agrega un nuevo Layer
  - Agregado de archivos a utilizar
- **Image**
  - Bloques estáticos construidos a través de Dockerfiles
  - Distros de Linux proveen base Images (Ubuntu, Debian, Centos, Alpine, etc.)
  - Se pueden reutilizar para construir nuevas Images



Docker | Conceptos Básicos (II)

- **Container**
  - Ejecución de comandos dentro de una Image
  - Kernel compartido entre Host OS y Container
  - Al terminar el comando a ejecutar,

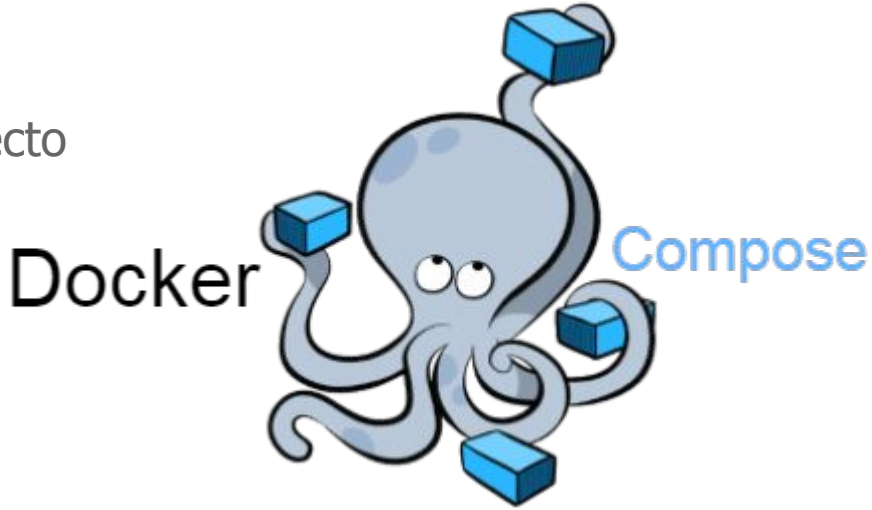
    el container deja de existir
  - No proveen persistencia
- **Volume**
  - Directorios compartidos entre el Host OS y container
  - Necesarios para dar persistencia a aplicaciones

Docker | Dockerfile

```
FROM centos:latest
WORKDIR /tmp

# Install wget to be able to download OpenMPI
RUN yum update -y && \
    yum groupinstall "Development Tools" -y && \
    yum install wget vim -y

# Compile and install OpenMPI
RUN wget https://www.open-mpi.org/software/ompi/v3/openmpi-3.0.0.tar.gz && \
    tar -xvzf openmpi-3.0.0.tar.gz &&\
    cd openmpi-3.0.0 && \
    ./configure --prefix=/opt/openmpi-3.0.0 && \
    make -j4 && make install && \
    cd /tmp && rm -rf /tmp/openmpi-3.0.0*

COPY env /opt/env
```

Docker Compose

- Desarrollado en Python
- Administración de múltiples *containers*
  - DNS Resolution
  - *Containers* en la misma red por defecto
- Configuración a través de YAMLs
  - Intuitivo
  - Mayoría de los comandos Docker disponibles



Docker Compose | docker-compose.yml

```yaml
web:
 image: nginx:latest
 ports:
   - "8080:80"
 volumes:
   - ./code:/code
   - ./nginx.conf:
            /etc/nginx/conf.d/default.conf
 depends_on:
   - php

php:
 build: .    #Requires a Dockerfile
 volumes:
   - ./code:/code
 environment:
   XDEBUG_CONFIG: remote_host=
                  host.docker.internal
 depends_on:
   - mysql
```

```yaml
mysql:
 image: mysql:latest
 ports:
   - "3306:3306"
 environment:
   - MYSQL_ROOT_PASSWORD=password

phpmyadmin:
 image: phpmyadmin/phpmyadmin
 depends_on:
   - mysql:db
 ports:
   - "8082:80"
 environment:
   - PMA_USER=root
   - PMA_PASSWORD=password
   - PHP_UPLOAD_MAX_FILESIZE=100MB
```

Bibliografía

- M. Van Steen, A. Tanenbaum: Distributed Systems. 3rd Edition. Pearson Education, 2017.
  - Cap. 1.3 Types of Distributed Systems
  - Cap. 2.1 - Architectures, Architectural Styles
  - Cap. 3.2 - Virtualization
  - Cap. 4 - Communication
- G. Coulouris, J. Dollimore, t. Kindberg, G. Blair: Distributed Systems. Concepts and Design, 5th Edition, Addison Wesley, 2012.
  - Cap. 7.7.1 System virtualization
- W. Stallings, Data and Computer Communication, 8th Edition.
  - Cap. 2 - Protocol Architecture,TCP/IP, and Internet-Based Applications